# Computational Physics Laboratory report

Stefano Ge

Winter semester 2025

## Contents

**0 Introduction** **2**
0.1 Considerations on the C programming language . . . . . 3

**1 Error analysis** **4**
1.0.1 Approximating exponentials . . . . . 4
1.2 Floating-point arithmetic and roundoff errors . . . . . 5
1.2.1 Computing the Basel problem . . . . . 5
1.3 Error propagation and condition number . . . . . 7
1.3.1 Computing statistical momenta . . . . . 7
1.3.2 Condition number: study of a simple algorithm . . . . . 7

**2 Linear systems** **7**
2.1 Forward- and back-substitution . . . . . 7
2.2 LUP Decomposition . . . . . 7
2.3 Conditioning of linear systems . . . . . 8
2.4 Overdetermined linear systems . . . . . 9
2.4.1 Least squares through normal equations . . . . . 9
2.4.2 Fitting of Kepler's Law . . . . . 10
2.4.3 Fitting of exp(sine) function . . . . . 10
2.5 QR decomposition . . . . . 11
2.5.1 Least squares through QR . . . . . 12
2.5.2 QR decomposition for eigenvalues and eigenvectors . . . . 13

**3 Interpolation** **13**
3.1 Polynomial interpolation by solving the Vandermonde matrix linear system . . . . . 13
3.2 Lagrange-Waring interpolation . . . . . 13
3.2.1 Evenly spaced nodes . . . . . 13
3.2.2 Chebyshev nodes . . . . . 14
3.2.3 Chebyshev vs evenly spaced nodes: error study . . . . . 14
3.2.4 Polynomial interpolation over the real line with Chebyshev nodes . . . . . 14
3.2.5 Chebyshev nodes interpolation of non-smooth functions . 15

**4 Roots of nonlinear equations** **16**
4.1 Bisection method . . . . . 16
4.2 Newton method . . . . . 16
4.2.1 Convergence rate study . . . . . 16
4.2.2 Which root gets found by Newton's method? . . . . . 18
4.2.3 Eros' true anomaly . . . . . 19
4.3 Secant method . . . . . 19
4.4 Inverse quadratic interpolation . . . . . 20

**5 Numerical integration** **20**
5.1 Newton-Cotes formulas . . . . . 20
5.1.1 Trapezoidal rule . . . . . 21
5.1.2 Simpson's rule . . . . . 22
5.2 Gauss-Legendre rule . . . . . 23
5.2.1 Nodes and weights of Gauss-Legendre rule . . . . . 23
5.2.2 Integrals with Gauss-Legendre rule . . . . . 24
5.3 Double exponential rule . . . . . 25
5.3.1 Comparison of double exponential and Gauss-Legendre rules 25
5.3.2 Improper integrals with double exponential rule . . . . . 27

**6 Ordinary differential equations** **28**
6.1 Euler's method . . . . . 28
6.1.1 First order application . . . . . 28
6.1.2 Second order application . . . . . 28
6.2 Runge-Kutta methods . . . . . 29
6.2.1 Improved Euler's and Runge-Kutta fourth order methods 29
6.2.2 Non-linear equation application of Runge-Kutta . . . . . 29
6.2.3 Predator-prey model . . . . . 30

**7 Schrödinger equation** **31**
7.1 Time-dependant Schrödinger equation . . . . . 31
7.1.1 Gaussian packet in an infinite well . . . . . 32
7.1.2 Moving gaussian packet . . . . . 32
7.1.3 Finite barrier scattering . . . . . 32
7.1.4 Pöschl-Teller potential . . . . . 33
7.2 Time-independant Schrödinger equation . . . . . 34

**Bibliography** **35**

# 0 Introduction

This is the report of the Computational Physics Laboratory experienced by Stefano Ge in winter semester of the 2025/2026 academic year. Aside from the last section on the Schrödinger equation, the laboratory mainly covered foundational topics in numerical mathematics.

This report contains results and remarks on the numerical exercises assigned during lectures. We do not explain throughly the assignments' text, found in [Dal25], and we already assume the reader familiar with related theory.

## 0.1 Considerations on the C programming language

The numerical exercises are all programmed in C, as by choice of myself. Although valid alternatives have been developed, the C/C++ languages remain the standard of high-performance scientific computing. Compared to Python, they provide better performance and low-level control on the program.

They however have some significant disadvantage: they are harder to use, leaving many responsibilities to the programmer, and are often meant to be used with the programmer programming its custom libraries for common mathematical routines. Also, C lacks compelling plot libraries, which forced us to use Gnuplot. As a consequence, programming in C takes much more time than in Python. Because of this, much more time had to be reserved to programming than expected, and the overall experience involved less numerical mathematics than intended. Aside from Schrödinger equation, almost no other optional exercise had been carried out. We do not have time to add more research on numerical mathematics or references to books aside from the lecture notes.

On the other hand, the experience taught us how to write very good C programs, a skill which is not trivial nor common among Physics undergraduates in these days. Due to the relevance of programming in our experience, we write below some lessons that we learned.

#### Programming lessons and skills learnt during the course

- Use of Gnuplot. C program →plotting library →procedurally generated Gnuplot script workflow.
- Use of CMake. Organisation of large library with header and source files. Organisation of folder tree for programming.
- Use of Git and GitHub for versioning.
- Consistent convention for clear and readable naming of variables and functions. Understanding of when it is ideal to use a verbose syntax, typical of corporate-level programming, and when a concise mathematicla syntax should be used.
- Implementation of efficient and versatile data structures like `Array` and `Matrix`.

We chose C over C++ simply because did not have enough experience on the language to use it efficiently. C++ has many features, but it is however much harder to master fully than C.

The experience with C in this course, however, has taught much on good programming practices that would translate very well in C++. In the following paragraph we list a few reasons why we would use C++ if we were to rewrite our code.

#### Preference for C++

- Classes to build our containers more handily. The possibility of overloading operators in order to make sense of addition and multiplication of matrices and arrays.

- Less reliance on pointers thanks to dereferencing. Safer memory allocation thanks to unique and shared pointers, constructors and destructors.
- The ease in writing generic classes and functions for different data types (`double`, `float`, `complex`, etc).

# 1 Error analysis

### 1.0.1 Approximating exponentials

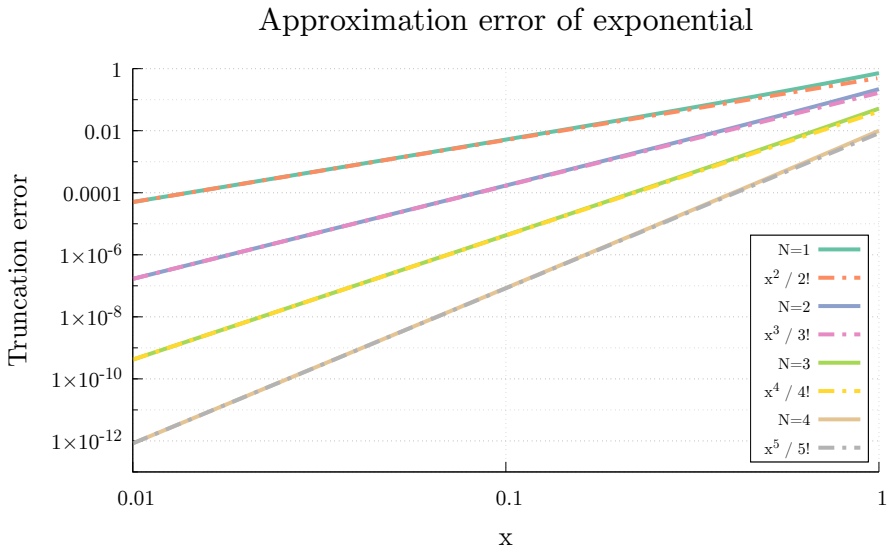



Figure 1: Truncation errors when approximating $e^x$ through the truncated power series $\Sigma_0^N$. $N$ is the degree of the polinomyal used for the approximation. The plot of $x^{N+1}/(N+1)$ has been added for reference.

**Results**

**Remarks**

- Horner's method was used in polynomial evaluation, because it is faster and reduces roundoff errors. A naive implementation of a polynomial computation requires more operations ($O(N^2)$) and, for small values of $x$, adds together numbers of order 1 and $x^N$, while Horner's method is $O(N)$ and keeps the additions only between terms of order 1 and x.
- By Taylor's theorem, if $\hat{f}(x)$ is the power series of $e^x$ truncated after the N-th term, the error is of order $O(x^{N+1})$ as x approaches 0. Computing an additional term of the series shows that:

$$|e^x - \hat{f}(x)| = \frac{x^{N+1}}{(N+1)!} + o(x^{N+1})$$

It is then not surprising that the approximation error in Figure 1 is indistinguishable from $x^{N+1}/(N+1)$ for small values of x. For larger values, the contribution of higher order terms in the series becomes ever more important, and hence the divergence from $x^{N+1}/(N+1)$.

## 1.2 Floating-point arithmetic and roundoff errors

### 1.2.1 Computing the Basel problem

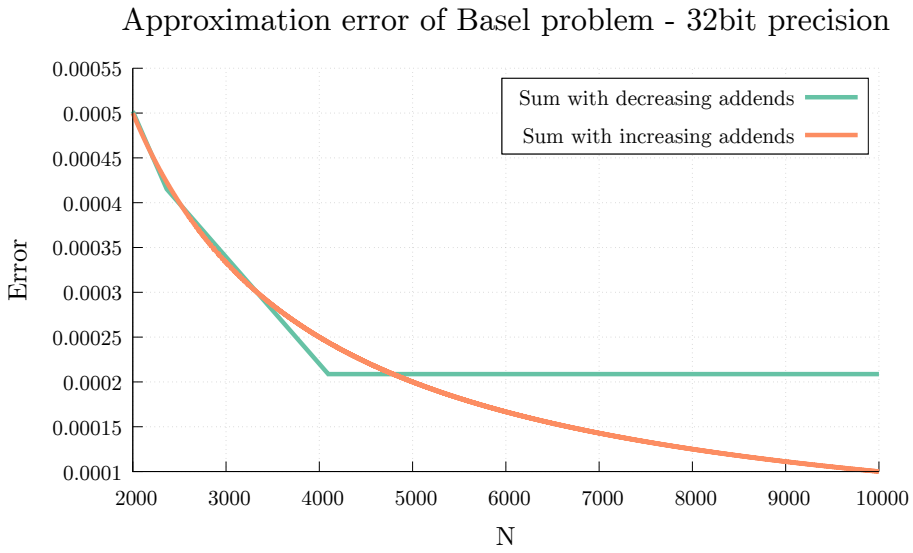



Figure 2: Approximation error when computing $\pi^2/6$ with the Basel problem truncated series $\Sigma_0^N \frac{1}{n^2}$, with single precision floating variables. Here N is the number of terms used when calculating the series.

**Results**

**Remarks**

- The algorithm has been implemented with two mathematically equivalent forms, which, however, resulted in remarkably different numerical results. In the first the terms of the series are summed in increasing order of the summation index, i.e. decreasing of the value of the addends. Since small addends are summed to the accumulating partial sum, it gets to a point where the next addend is smaller than $\epsilon_{mach} \cdot sum \approx \epsilon_{mach}$, so the addition of it and subsequent values does not change the value of the partial sum.
  This can be seen in the error plots in Figure 2 and 3, since the error stops decreasing and remains constant after $1/\mathrm{N}^2 = \epsilon_{\mathrm{mach}}$, which is $2^{-24}$ for single precision ($\mathrm{N} = 4096$) and $2^{-53}$ for double precision ($\mathrm{N} \approx 9.5 \times 10^7$).

  In the second implementation the terms of the series are summed in decreasing order of the summation index, i.e. increasing the value of the

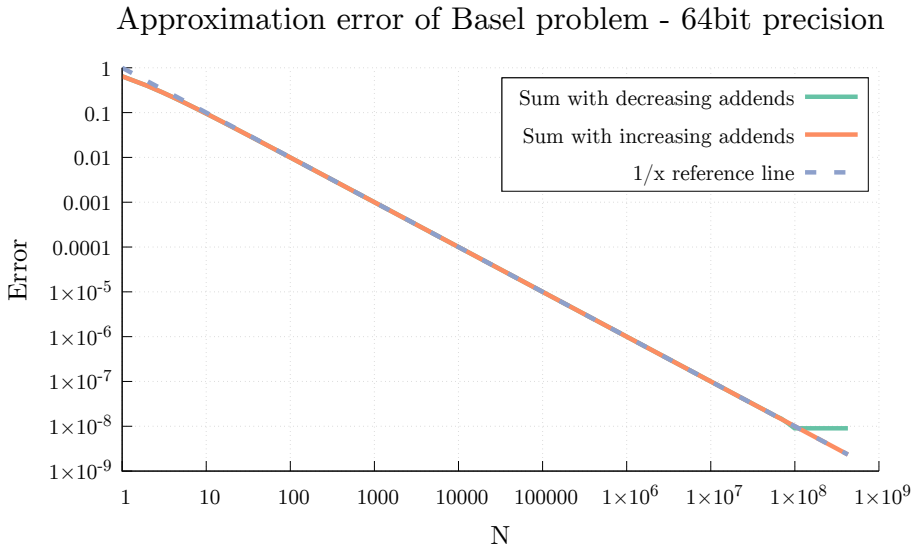



Figure 3: Same as above, but now using double precision floating variables. the plot of $1/x$ has been added for reference.

addends. Therefore, the aforementioned effect is not seen, and the numerical results correspond to the mathematical expectation, in the range considered.

- The truncation error of this algorithm is asymptotically equivalent to $k/N$, with k real constant. This can be from the integral associated to $\Sigma_{n=0}^{\infty}$:

$$\int_N^{\infty} \frac{1}{n^2} = \frac{1}{N}$$

therefore,

$$\Sigma_{n=N}^{\infty} \sim_{N\to\infty} \frac{1}{N}$$

which is confirmed from the agreement between the computed values and the $1/x$ reference line in Figure 3.

### 1.3 Error propagation and condition number

#### 1.3.1 Computing statistical momenta

#### 1.3.2 Condition number: study of a simple algorithm

## 2 Linear systems

### 2.1 Forward- and back-substitution

**Results**

$$\begin{bmatrix} -2 & 0 & 0 \\ 1 & -1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} -4 \\ 2 \\ 1 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 2 \\ 0 \\ -5 \end{bmatrix}$$

$$\begin{bmatrix} 4 & 0 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ -1 & 4 & 4 & 0 \\ 2 & -5 & 5 & 1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} -4 \\ 1 \\ -3 \\ 5 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} -1 \\ -1 \\ 0 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & -1 & -2 \\ 0 & 0 & 3 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ 1 \\ 6 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 2 \\ -5 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 & 0 & 6 \\ 0 & -1 & -2 & 7 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 5 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 4 \\ 1 \\ 1 \\ 5 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} -3.\overline{3} \\ 8 \\ -1 \\ 1 \end{bmatrix}$$

The results obtained through my routine were verified using LAPACK.

### 2.2 LUP Decomposition

**Results**

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0.5 & 0.\overline{3} & 1 \end{bmatrix} \begin{bmatrix} 4 & 8 & 2 \\ 0 & -3 & 8 \\ 0 & 0 & 0.\overline{3} \end{bmatrix}$$

Determinant is 4.

$$\begin{bmatrix} 6 & -2 & -4 & 4 \\ 3 & -3 & -6 & 1 \\ -12 & 8 & 21 & -8 \\ -6 & 0 & -10 & 7 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ -0.25 & 0.25 & 1 & 0 \\ -0.5 & -0.5 & -0.857143 & 1 \end{bmatrix} \begin{bmatrix} -12 & 8 & 21 & -8 \\ 0 & -4 & -20.5 & 11 \\ 0 & 0 & 4.375 & -3.75 \\ 0 & 0 & 0 & 2.28571 \end{bmatrix}$$

Determinant is -480.

$$\begin{bmatrix} 1 & 4 & 5 & -5 \\ -1 & 0 & -1 & -5 \\ 1 & 3 & -1 & 2 \\ 1 & -1 & 5 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0.8 & 1 & 0 \\ -1 & -0.2 & -0.\overline{6} & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 5 & -1 \\ 0 & 5 & 0 & -4 \\ 0 & 0 & -6 & 6.2 \\ 0 & 0 & 0 & -2.\overline{6} \end{bmatrix}$$

Determinant is 80.

Matrix $\mathbf{A} = \mathbf{T}(3,-1)\mathbf{R}(\pi/5)\mathbf{T}(-3,1)$ :

$$A = \begin{bmatrix} 2.3 & 1.6 & -2.3 \\ -1.1 & 0.48 & 2.7 \\ 0.5 & 0.\overline{3} & -0.\overline{6} \end{bmatrix}$$

$$\mathbf{b} = \mathbf{Az} = \mathbf{A}\begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 5.45 \\ 1.51 \\ 1.5 \end{bmatrix}$$

Factorization of $\mathbf{A}$ is:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ -0.471 & 1 & 0 \\ 0.217 & -0.00857 & 1 \end{bmatrix}\begin{bmatrix} 2.31 & 1.59 & -2.34 \\ 0 & 1.22 & 1.64 \\ 0 & 0 & 0.354 \end{bmatrix}$$

## 2.3 Conditioning of linear systems

**Results** In the case study proposed, first we construct numerically:

$$\mathbf{b} = \mathbf{Ax} = \begin{bmatrix} -\epsilon & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Where $\epsilon$ is a real parameter. We then apply the LU decomposition (without pivoting) to solve for x given $\mathbf{A}$ and the $\mathbf{b}$ just computed.

For $\epsilon = 10^{-12}$ we find:

$$\mathbf{A} = \mathbf{LU} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-12} & 1 \end{bmatrix}\begin{bmatrix} -\text{e}{-12} & 1 \\ 0 & 1\text{e}12 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2.2\text{e}{-5} \\ 0 \end{bmatrix}$$

As a test, we find correctly:

$$\mathbf{A} - \mathbf{LU} = \mathbf{0}$$

However, if we set $\epsilon = 10^{-12}$, we find:

$$\mathbf{A} = \mathbf{LU} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-20} & 1 \end{bmatrix}\begin{bmatrix} -\text{e}{-20} & 1 \\ 0 & 1\text{e}20 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

$$\mathbf{A} - \mathbf{LU} = \begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}$$

**Remarks**

- The exact LU-factorization of $\mathbf{A}$ is:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ -\epsilon^{-1} & 1 \end{bmatrix}\begin{bmatrix} -\epsilon & 1 \\ 0 & \epsilon^{-1} - 1 \end{bmatrix}$$

  If $\epsilon = 10^{-20} \ll \epsilon_{\text{mach}}$, in the bottom-right entry of $\mathbf{U}$ only $\epsilon^{-1}$ gets stored, since the relative variation of adding -1 to $\epsilon^{-1}$ is less than $\epsilon_{\text{mach}}$. But

when $\mathbf{L}$ and $\mathbf{U}$ get multiplied together, in the addition resulting into the bottom-right entry of $\mathbf{A}$ two $\epsilon^{-1}$ terms cancels out, and the exact result is the -1 that is lost in numerical LU-decomposition.

- Exact $\mathbf{b}$ is:
$$\begin{bmatrix} 1-\epsilon \\ 0 \end{bmatrix}$$
but $\epsilon$ does not get registered if it is lower than $\epsilon_{\text{mach}}$. Because the condition number of the matrices $\mathbf{L}$ and $\mathbf{U}$ is very high ($\epsilon^{-2}$ in $\infty$-norm), the tiny variation of $\mathbf{b}_{\text{calc}}$ over $\mathbf{b}_{\text{exact}}$ is allowed to propagate tremendously and results in the wrong result seen before.

  Doing the calculations shows that the error on $\mathbf{U}$ as no impact on $\mathbf{b}_{\text{calc}}$, and that the final error on $\mathbf{b}_{\text{calc}}$ is due entirely on the ill-conditioning of the linear systems $\mathbf{Lz} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{z}$.

- By contrast, the condition number of the starting matrix $\mathbf{A}$ is only $\approx$4. Therefore, in principle it should be possible to solve the system for $\mathbf{x}$ even if $\mathbf{b}$ is slightly nudged: the problem is the stability of the algorithm adopted.

  LU-decomposition without row-pivoting is much less numerically stable than LUP-decomposition with row-pivoting where the maximum value entry in a column is selected as the pivot.

  Indeed, if we perform LU-decomposition on the following matrix, obtained by permuting the two rows of matrix $\mathbf{A}$, as prescribed by row-pivoting, we find the correct results:
$$\mathbf{PA} = \begin{bmatrix} 1 & -1 \\ -\epsilon & 1 \end{bmatrix} = \mathbf{LU} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-20} & 1 \end{bmatrix} \begin{bmatrix} ta1 & -1 \\ 0 & 1 \end{bmatrix}$$
$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\mathbf{A} - \mathbf{LU} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

## 2.4 Overdetermined linear systems

### 2.4.1 Least squares through normal equations

**Results** After implementing a routine for finding a the vector $\mathbf{X}$ that best fits an overdetermined linear system, we tested it on the following linear system:
$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ -5 \\ 6 \end{bmatrix}$$
and found:
$$\mathbf{X} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

### 2.4.2 Fitting of Kepler's Law

We want to extrapolate Kepler's third law from experimental data through the algorithm just introduced. First we linearize the following relationship by taking the logarithm at both sides:

$$\mathrm{T} = \mathrm{CD}^{\alpha} \implies \log(\mathrm{T}) = \log(\mathrm{C}) + \alpha \log(\mathrm{D})$$

where T is the orbital periods of the planets, D is their average distance from the Sun and alpha and C are two costants to be determined through the method of least squares. This is equivalent to finding $x_1$ and $x_2$ that minimize:

$$\log(\mathbf{T}) = x_1\mathbf{1} + x_2\log(\mathbf{D}) = \begin{bmatrix}\mathbf{1} & \log(\mathbf{D})\end{bmatrix}\begin{bmatrix}x_1 \\ x_2\end{bmatrix}$$

where quantities in bold are vectors, leading to an overdetermined linear system.

**Results** We therefore find that the values that fit the aforementioned system are:

$$\alpha = 1.49865 \qquad C = 0.201522$$

where alpha is consistent with Kepler's law, which predicts $\alpha = 1.5$.

The graph containing the fitted curve and experimental data can be found in Figure 4

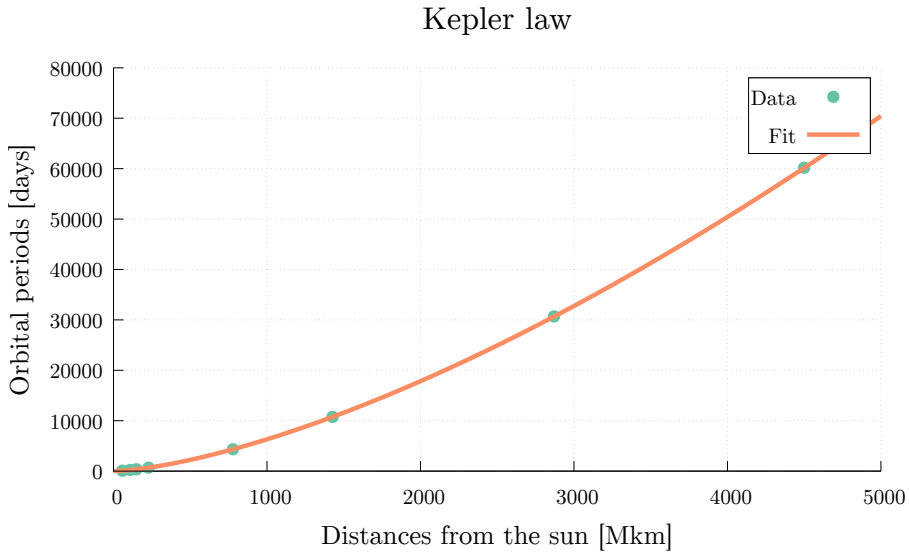



Figure 4: Orbital peirods vs distances from the Sun of the planets.

### 2.4.3 Fitting of exp(sine) function

**Results** We want to approximate the function $\exp(\sin(t-1))$, first through a Taylor-like polynomial:

$$y(t) = c_1 + c_2 t + ... + c_7 t^6$$

and then through a periodic Fourier-like approximation:

$$y(t) = d_1 + d_2 \cos(t) + d_3 \sin(t) + d_4 \cos(2t) + d_5 \sin(5t)$$

Results are shown in Figure 5 and in Figure 6.

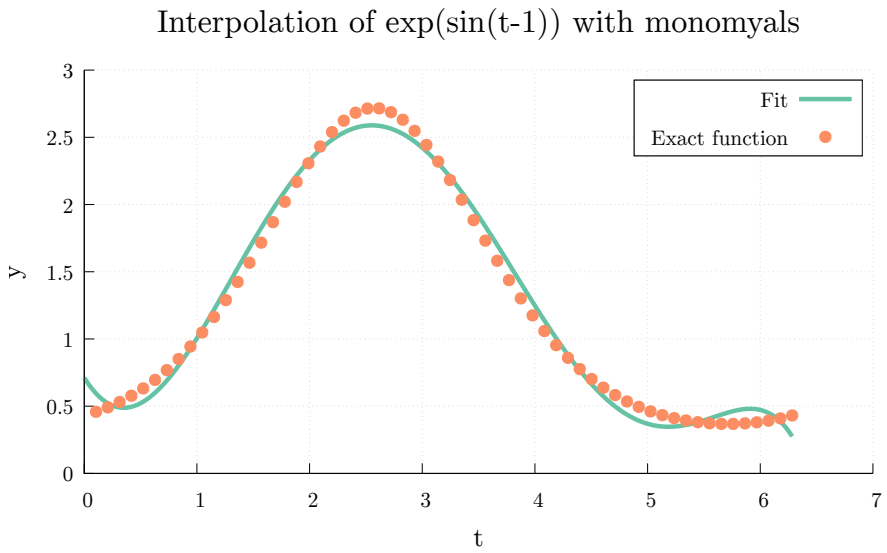



Figure 5: Least square fit of $\exp(\sin(t-1)$ with 6th degree polynomial

**Remarks**

- Not surpringly, the function is better approximated by the first terms of the Fourier basis than by a sixth degree polynomial. The Fourier basis is an orthonormal complete set of $L^2([0, 2\pi])$, therefore it is guaranteed to be the best approximation in $L^2$ norm, among expansions on sets spanned by it. Moreover, sines and cosines are themselves periodic and are thus well suited for approximating a periodic function like $\exp^{\sin(t-1)}$, producing a nice approximation over the whole domain $\mathbb{R}$. On the other hand, monic monomyals, though complete, are neither an orthonormal set of $L^2([0, 2\pi])$ nor are well suited for approximating periodic functions, as they do blow up at infinity and do not approximate well outside of the points employed in the fit.

## 2.5 QR decomposition

In this section we implement algorithms to find the QR decomposition of any matrices. First the modified Gram-Schmidt procedure (MGS).

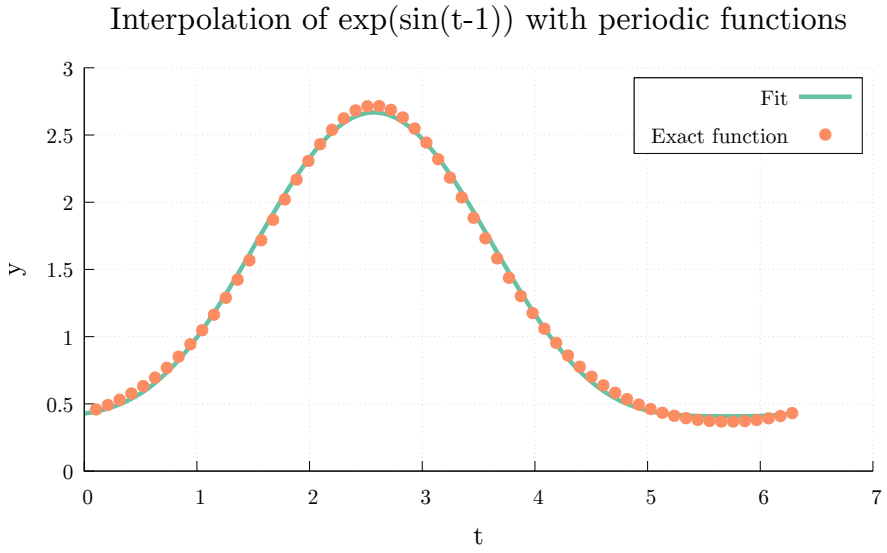



Figure 6: Least square fit of $\exp(\sin(t-1)$ with Fourier periodic functions

**Results** We test the modified Gram-Schmidt procedure on the following matrices:

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 0.707 & 0.408 \\ 0.000 & 0.816 \\ -0.707 & 0.408 \end{bmatrix} \begin{bmatrix} 2.828 & -2.121 \\ 0 & 1.225 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix} = \begin{bmatrix} 0.333 & -0.104 & 0.937 \\ 0.667 & -0.677 & -0.312 \\ 0.667 & 0.729 & -0.156 \end{bmatrix} \begin{bmatrix} 6.000 & 9.667 & 9.333 \\ 0 & 2.134 & -5.726 \\ 0 & 0 & 0.312 \end{bmatrix}$$

$$\begin{bmatrix} 2 & -1 & 4 \\ 0 & 3 & 5 \\ 7 & 2 & -6 \\ 1 & -4 & 0 \end{bmatrix} = \begin{bmatrix} 0.272 & -0.241 & 0.733 \\ 0.000 & 0.559 & 0.617 \\ 0.953 & 0.179 & -0.233 \\ 0.136 & -0.773 & 0.163 \end{bmatrix} \begin{bmatrix} 7.348 & 1.089 & -4.627 \\ 0 & 5.368 & 0.752 \\ 0 & 0 & 7.418 \end{bmatrix}$$

Computing the product $\mathbf{A} = \mathbf{QR}$ gives us the matrices we have decomposed, confirming the correctness of the decomposition. Also we computed $\mathbf{Q}^\intercal\mathbf{Q}$ and we confirmed that the computed $\mathbf{Q}$'s are indeed ONC matrices.

### 2.5.1 Least squares through QR

We implemented QR algorithm to solve least-squares problem, and applied it to the problem of fitting the function $\exp^{\sin(4t)}$ with a 14th degree polynomial.

**Results**

#### 2.5.2 QR decomposition for eigenvalues and eigenvectors

# 3 Interpolation

## 3.1 Polynomial interpolation by solving the Vandermonde matrix linear system

Interpolating polynomials in the power basis by constructing the Vandermonde matrix and solving the related linear system is numerically unstable. Tha matrix thus constructed is likely to be ill-conditioned, due to the fact that its entries are the nth powers of x, and can become quickly extremely large or small as the system scales in size.

In our case study, we considered equidistant point in the interval [-1,1] generated by the function $\cos(kx)$.

In Figure AGGIUNGI REFERENCE we can see that the condition number scales exponentially with the number of points considered in the fit, which is also the size of the linear system.

AGGIUNGI FIGURA

In our study we always considered $4n + 1$ equidistant points, including the extrema of the interval -1 and 1. As a consequence, the middle point considered is always (0,1), since $\cos(0) = 1$. Therefore, we must have:

$$P_n(0) = a_0 + a_1 \cdot 0 + ... + a_n \cdot 0 = a_0 = 1$$

CERCA DI CAPIRE COME MOSTRARE I VALORI

If we were operating in exact arithmetic, this would produce a value of $a_0$ always equal to 1, since it would lead to a linear system where a row of the Vandermonde matrix (the one corresponding to point x=0) which is equivalent to equation $a_0 = 1$. However, the LUP decomposition algorithm contaminates several times the aforementioned row of the linear system, relying later on cancellation of possibly large values to recover the information of $a_0 = 1$. Therefore, in finite-precision arithmetic the value of $a_0$ can differ from 1, as we can see ... . This is closely related to the ill-conditioning of the Vandermonde matrix.

## 3.2 Lagrange-Waring interpolation

### 3.2.1 Evenly spaced nodes

**Results** In Figure ADD 4 GRAPHS AND REFERENCES we show the polynomials resulting from the Lagrange-Waring interpolation of the the functions $\log(x)$, $tanh(x)$, $cosh(x)$ and $|x|$, using equally spaced nodes.

The weights of the points were evaluated through the analytic formula for equally spaced weights. Some lines of code have been added to the code in order to show that the special formula agrees with the generic one.

**Remarks**

- The Lagrange-Waring interpolation implementation has been organised around a struct, `BarFit`, which stores points, weights, etc.

  This OOP-style design is flexible, and allows reusage of the weights already calculated for other functions.

### 3.2.2 Chebyshev nodes

**Results** We have tested our routine for Lagrange interpolation using Chebyshev nodes of the second kind by fitting the function $cosh(sin(x))$ using 40 points in the interval [0, $2\pi$].

The interpolating polynomial, together with the nodes, can be visualized in ADD REFERENCE

In the testing program we also included a a comparison between the weights calculated through the analytical formula for Chebyshev nodes and the generic nodes formula. It confirmed that the weights are the same aside from roundoff errors.

### 3.2.3 Chebyshev vs evenly spaced nodes: error study

We study the convergence of Lagrange intrpolation wit equidistant an Chebyshev points for the functions $1/(25x^2 + 1)$, $\tanh(5x + 2)$, $\cosh(\sin(x))$ and $\sin(\cosh(x))$ over the inerval [-1:1].

The results are shown in ADD REFERENCS AND FIGURES

**Remarks**

- Interpolation theory has proven that, for analytical functions bounded in the interpolation interval, $||f - p||_\infty \leq CK^n$ for some constants C and K, i.e. that Lagrange's interpolation with Chebyshev nodes converges at least exponentially. In our plot we see this exact pattern. We can even estimate the constant $K$:

| f(x) | K |
|---|---|
| $1/(25x^2 + 1)$ | 1.22 |
| $\tanh(5x + 2)$ | 1.39 |
| $\cosh(\sin x)$ | 5.70 |
| $\sin(\cosh x)$ | 4.81 |

- When we use equidistant points as nodes convergence is not guaranteed for every analytical and bounded functions. Instead, as the number of nodes grows, $||f - p||_\infty$ can grow exponentially, that is Runge's phenomenon. We observe this phenomenon in our plot.

### 3.2.4 Polynomial interpolation over the real line with Chebyshev nodes

By means of a change of variable we now interpolate the function $1/(z^2-2z+2)$ over an unbounded interval [-$\infty$, +$\infty$]. We used Chebyshev nodes of the first kind to avoid evaluation at x = $\pm\infty$.

As change of variable we used the function $\phi(x) = \frac{2x}{1-x^2}$ which maps the interval [-1, 1] to the real line.

Results are shown in ADD REFERENCE AND FIGURE

### 3.2.5 Chebyshev nodes interpolation of non-smooth functions

Polynomial interpolants on Chebyshev nodes are guaranteed to converge exponentially to the reference function if the latter is analytic, as the number of nodes approaches infinity. If it is not, convergence might not be guaranteed or it might be slower.

We can observe this slowdown by studying the convergence for the family of functions $|x|^m$. These functions are not smooth on the interval [-1, 1]. They can be differentiated $m-1$ times, but not $m$ times, since $|m|$ is not differentiable in $x = 0$. Hence they are not analytic. The results are shown in ADD REFERENCE AND FIGURES

**Remarks**

- As previously said, $|x|^m$ is not analytical. However, their derivatives are continuously differentiable up to $f^{(m-2)}$, thus absolutely continuous. $f^{(m-1)} = C|x|$, which is Lipschitz continuous and as such it is absolutely continuous. On the other hand:

$$f^{(m)}(x) = \begin{cases} +C, & x > 0 \\ -C, & x < 0 \end{cases}$$

  for some C>0. This function is monotonic, hence it is of bounded variation. This proves a theorem that states that if the above hypotheses are verified (absolute continuity up to $m-1$-th derivative and bounded variation of $m$-th derivative), then the Chebyshev polynomial interpolant converges to the function in sup-norm as $\mathcal{O}(n^{-m})$, as the number of nodes n approaches infinity.

- To compare this prediction with our results, we interpolated the sup-norm error using algorithm from ADD REFERENCES By choosing $Y = \log_{10}(||f-p||_\infty) = \log_{10}(c) - K\log_{10}(n)$ and $X = \log_{10}(n)$ we interpolated the relation:

$$Y = C - K\,X$$

  where -K is the exponent in the asymptotic behaviour $\mathcal{O}(n^{-m})$ of $||f-p||_\infty$. The fitting lines can are visualised in ADD REFERENCE as dashed lines. Our results are reported in the following table and are in accordance to the prediction above:

| m | K |
|---|---|
| 1 | 1.04 |
| 3 | 3.13 |
| 5 | 5.15 |
| 7 | 7.26 |
| 9 | 9.41 |
| 11 | 11.7 |

# 4 Roots of nonlinear equations

## 4.1 Bisection method

In order to test our routine for rootfinding with the bisection method, we searched for the roots of the function $x^3 - 7x^2 + 14x - 6$ in the interval [0, 1] and found $x = 0.5858$. The plot (which we omitted from the report, but can be visualized during code execution) of the function and the found root confirms that it is the only root in the interval.

Given an iterative rootfinding algorithm, if we define $\epsilon_n = x_n - r$, where $r$ is the root that we are going to find and $x_n$ is the root estimate at step n, then if

$$\lim_{n \to +\infty} \left| \frac{\epsilon_{n+1}}{\epsilon_n^q} \right| = C \tag{1}$$

for $q \geq 1$ and $0 < C < \infty$, we say that the algorithm converges to the root with order of convergence $q$ and asymptotic constant $C$.

It is very easy to see that for the bisection algorithm $q = 1$ and $C = \frac{1}{2}$. Under this assumption, we get the asymptotic relation

$$|\epsilon_{n+1}| \approx |\frac{1}{2}\epsilon_n|$$

which is the asymptotic recurrence relation of a geometric sequence $\epsilon_n = K(\frac{1}{2})^n$.

Defining $a_n = -\log_{10}(|\epsilon_n|)$ we get:

$$a_n = K' + n \log_{10}(2)$$

In ADD FIGURE AND REFERENCE we plotted the $a_n$ as a function of $n$. The curve is a line, confirming that the order of convergence is 1. Fitting the curve with a line through least squares yields a slope of 1.992, which implies an asymptotic error constant $C = 0.5018$, in accordance with bisection method's convergence theory.

As a termination condition of the loop in the algorithm we check if:

$$|x_1 - x_2| <= \text{tol} + \epsilon_{\text{mach}} \max(|a|, |b|)$$

Where $|a|$ and $|b|$ are the boundaries of the starting interval and $x_1$, $x_2$ are the boundaries of the interval at current step.

`tol` can be specified by the caller or instead be $10\epsilon$mach if the caller inserts the macro `DEF_TOL` as parameter.

$\epsilon_{\text{mach}} \max(|a|, |b|)$ is added because it is approximately the distance between the largest of the two starting boundaries (in absolute value). The algorithm is not always able to bisect the algorithm further when it is of the order of the quantity above, so it is a safety check.

## 4.2 Newton method

### 4.2.1 Convergence rate study

We seek to check the correctness of our routine and study the convergence rate of Newton's method by applying it to the functions $f_1(x) = x^2 - \exp(-x)$, $f_2(x) = \tan(x) - 2x$ and $f_3(x) = \exp(x + 1) - x - 2$.

By calculating $a_n = -log_{10}|x_n - r|$ we can calculate how many digits of precision we are gaining at each step, and estimate order of convergence and asymptotic error constant.

**Results** In ADD FIGURES AND REFERENCES we plotted the functions and the roots found.

In table 1 we quote the roots found and the numerical estimates for the values of $q$ and $C$.

$q$ is estimated as the asymptotic value of $a_{n+1}/a_n$, since it holds $a_{n+1} \sim qa_n$.

$C$ is estimated as

$$|\epsilon_{n+1}/\epsilon_n^q|$$

where we use for $q$ the value that we calculated analytically (see Remarks).

| f(x) | Root | ≈q | ≈C |
|---|---|---|---|
| $x^2 - \exp(-x)$ | 0.703 | 2.08 | 0.40 |
| $\tan(x) - 2x$ | 0.000 | 3.07 | 0.67 |
| | 1.166 | 1.92 | 3.38 |
| $\exp(x+1) - x - 2$ | -1.000 | 1.06 | 0.50 |

**Remarks**

- If a root $r$ of a function $f(x)$ satisfies $f'(r) \neq 0$ and $f''(r) \neq 0$, then Newton's method's convergence order is quadratic for that root and the asymptotic error constant is:

  $$C = \frac{1}{2}\left|\frac{f''(r)}{f'(r)}\right|$$

  That is the case for the root of $f_1$ and for root $x = 1.166$ of $f_2$. In both cases the estimated convergence order is close to 2 and the estimated asymptotic error constant agrees with the one calculated analitically by 4+ significant digits. Hence, we do not include separately the analitical values in the report and we refer the reader to the program `newton_method_error_study.c`.

- If the root is not simple, i.e. $f'(r) = 0$, but $f''(x) \neq 0$, then it can be shown that convergence is linear with asymptotic error constant $C = 0.5$. Root the root of $f_3$ satisfies this condition. In our analysis we see that most ratios $a_{n+1}/a_n$ agree wih 0.5, except for the last few values.

  This is the same order of convergence and asymptotic error constant of bisection method. However, in bisection method only a single function evaluation is required at each step, while in Newton method both the function and its derivative must be evaluated. Thus, in the case $f'(r) = 0$, for the same precision gained at each step, Newton method is twice as expensive, compared to bisection method, if we assume the evaluation of the derivative to be comparable to evaluation of the function in computational cost.

- For root $r = 0$ of $f_3$ the estimated order of convergence is about 3. Seeing that for this function $f''(r) = 0$, this prompted us to check with a small computation whether $f''(r) = 0$ implies order of convergence 3. We start from the relation:

$$\epsilon_{k+1} = \epsilon_k - \frac{f(r+\epsilon_k)}{f'(r+\epsilon_k)}$$

which is a rewrite of Newton's algorithm. Expanding with Taylor's:

$$\epsilon_{k+1} = \epsilon_k - \frac{f(r) + f'(r)\epsilon_k + \frac{1}{2}\epsilon_k^2 f''(r) + \frac{1}{6}\epsilon_k^3 f'''(r) + O(\epsilon_k^4)}{f'(r) + f''(r)\epsilon_k + \frac{1}{2}\epsilon_k^2 f'''(r) + O(\epsilon_k^3)} =$$

$$= \epsilon_k - \frac{f'(r)\epsilon_k + \frac{1}{6}\epsilon_k^3 f'''(r) + O(\epsilon_k^4)}{f'(r) + \frac{1}{2}\epsilon_k^2 f'''(r) + O(\epsilon_k^3)} =$$

Collecting $f'(r)$:

$$= \epsilon_k \left[1 - \left(1 + \frac{1}{6}\epsilon_k^2 \frac{f'''(r)}{f'(r)}\right)\left(1 + \frac{1}{2}\epsilon_k^2 \frac{f'''(r)}{f'(r)}\right)^{-1}\right] + O(\epsilon_k^4) =$$

By doing some simple algebra:

$$= \frac{1}{3}\epsilon_k^3 \frac{f'''(r)}{f'(r)} + O(\epsilon_k^4)$$

i.e. cubic convergence, with asymptotic error constant $C = \frac{1}{3}f'''(r)/f'(r)$. By executing `newton_method_error_study`, the reader can see that the numerically estimated and the analitically calculated one agree by 4 significant digits, after only 6 steps have been taken by Newton's method.

### 4.2.2 Which root gets found by Newton's method?

Newton method, in general, is not globally convergent. Moreover, if two or more roots are present, if convergent there is no guarantee that the root that gets found is the one "closest" to the starting value for the algorithm.

In this paragraph we study the relationship between the starting value and the root that gets found by the method. We use as case study the function $f(x) = x^{-2} + \sin(x)$ and as starting values for the algorithm $x = 1, 2, ..., 7$.

In ADD FIGURE AND REFERENCES we have plotted $f(x)$, along with the starting values and the related roots that get found.

We see that the initial guesses 2 and 5 produce return roots that are not the closest to the starting points. In order to understand what is going on, we have drawn the tangent line to the graph at each of those two initial guesses. Newton method approximates the function to that line at the first iteration. We can see from the graph the tangent lines drawn intersect the x axis "far away" from the starting guess, since at both points the function has a slope small in absolute value.

If we include bracketing in our algorithm, then we can be sure that the root that gets found will be inside of the interval from which we started.

ADD FIGURE AND REFERENCE shows the root found inside the arbitrality chosen intervals [0.5, 2], [2, 5], [5, 8] and [8, 10].

### 4.2.3 Eros' true anomaly

As an application of Newton method, we seek to evaluate the *true anomaly* $\theta(t)$ of asteroid Eros, defined as:

$$\tan\frac{\theta}{2} = \sqrt{\frac{1+\epsilon}{1-\epsilon}}\tan\frac{\psi}{2}$$

For that, we need to evaluate first the *eccentric anomaly* $\psi(t)$ which can be found by solving the equaton:

$$f(\psi) = \psi - \epsilon\sin\psi - \frac{2\pi t}{\tau} = 0 \tag{2}$$

For asteroid Eros, we have *orbial period* $\tau = 1.7610$ and *elliptical eccentricity* $\epsilon = 0.2230$.

Derivative of equation 2 with respect to $\psi$ is:

$$f'(\psi) = 1 - \epsilon\cos\psi$$

Since $\epsilon < 1$, $f'(\psi) > 0$ for any $\psi$, the function is monotonic and we need not worry about finding multiple roots. Also, by plotting function 2 we see that it is reasonably well-behaved, and it might be even reasonable to expect global convergence. Nonetheless, we can also simply observe that for $\psi$ such that $f(\psi) = 0$:

$$\left|\psi - \frac{2\pi t}{\tau}\right| = |\epsilon\sin\psi| \leq \epsilon$$

that is, if we safeguard Newton method with bracketing and we take as starting interval $[\frac{2\pi t}{\tau} - \epsilon, \frac{2\pi t}{\tau} + \epsilon]$, then we are guaranteed to find the root.

In ADD FIGURE AND REFERENCES we have plotted the graph of $\theta(t)$ thus obtained. $\theta(t)$ was chosen to be in the interval $[0, 2\pi]$ to ensure continuity.

## 4.3 Secant method

We seek to test our implementation of the secant method and study its convergence rate by searching the roots of the same functions examined in 4.2.1.

**Results** In ADD FIGURES AND REFERENCES we plotted the functions and the roots found.

In table 1 we quote the roots found and the numerical estimates for the values of $q$, estimated as in 4.2.1.

| f(x) | Root | ≈q |
|---|---|---|
| $x^2 - \exp(-x)$ | 0.703 | 1.65 |
| $\tan(x) - 2x$ | 0.000 | 1.86 |
| | 1.166 | 1.58 |
| $\exp(x+1) - x - 2$ | -1.000 | 1.03 |

The values of $q$ quoted above must not be taken to be asymptotically accurate estimates. In contrast with our analysis of Newton method, the estimated

values were not settling around a value. With our precision we likely were not able to reach accurately the asymptotic behaviour; yet our rough estimation confirms a rate of convergence between linear and quadratic for the first 3 roots examined. In the case of $f_3(x)$, $f'(r) = 0$, therefore convergence is not guaranteed to be superlinear and appears to be linear instead.

**Secant method is the inverse interpolation method with degree 1** We prove the above statement as an exercise.

Suppose that we have calculated the approximations $x_k$ and $x_{k-1}$ for a root of the function $f(x)$ by using the inverse interpolation method of $m = 1$. Next approximation is $q(0)$, where $q(x)$ is a polynomial of degree 1 such that $q(f(x_k) = x_k$ and $q(f(x_{k-1}) = x_{k-1}$. But a polynomial of degree 1 is just a line. We can write it explicitely as:

$$q(y) - x_k = \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}(y - f(x_k))$$

By substituting 0 in place of y in the previous expression we find $x_{k+1} = q(0)$. We see that it is the same $x_{k+1}$ that would be calculated by the secant method.

## 4.4 Inverse quadratic interpolation

We now repeat the same study with the inverse quadratic interpolation method.

**Results** In ADD FIGURES AND REFERENCES we plotted the functions and the roots found.

In table 1 we quote the roots found and the numerical estimates for the values of $q$, estimated as in 4.2.1.

| f(x) | Root | ≈q |
|---|---|---|
| $x^2 - \exp(-x)$ | 0.703 | 1.6-2.0 |
| $\tan(x) - 2x$ | 0.000 | 1.6-2.0 |
| | 1.166 | 1.6-1.8 |
| $\exp(x + 1) - x - 2$ | -1.000 | 1.04 |

We confirm again an order of convergence between 1.6 and 2.0.

# 5 Numerical integration

## 5.1 Newton-Cotes formulas

In the following paragraphs we will test different integration rules on the integrals listed down here:

1) $f_1(x)$ : $\int_0^1 x \log(1+x)\, dx = \frac{1}{4}$

2) $f_2(x)$ : $\int_0^1 x^2 \tan^{-1} x\, dx = \frac{\pi - 2 + 2\log 2}{12}$

3) $f_3(x)$ : $\int_0^{\pi/2} e^x \cos x\, dx = \frac{e^{\pi/2} - 1}{2}$

4) $f_4(x)$ : $\int_0^1 \frac{\tan^{-1}(\sqrt{2+x^2})}{(1+x^2)\sqrt{2+x^2}} dx = \frac{5\pi^2}{96}$

5) $f_5(x)$ : $\int_0^1 \sqrt{x} \log(x)\, dx = -\frac{4}{9}$

6) $f_6(x)$ : $\int_0^1 \sqrt{1-x^2}\ dx = \frac{\pi}{4}$

We will sometimes refer to the definite integrals above simply by the integrand functions.

### 5.1.1 Trapezoidal rule

**Results** In ADD FIGURE AND REFERENCES, we plot the error $\Delta I$ between the exact value of the integrals and the approximation obtained using the trapezoidal rule, as a function of the number of subintervals used.

Since the spacing between the nodes $h$ is inversely proportional to the number of nodes $n$, if $\Delta I$ is asymptotically $O(h^m)$, then it must also be asymptotically $O(\frac{1}{n^m})$.

Given this, we can estimate $m$ by linearizing the asymptotic relationship between $\Delta I$ and $n$. $m$ will be the opposite of the slope of the line thus obtained. In Table 4 we list the values of $m$ found for every integral.

| $f$ | m |
|---|---|
| $f_1(x)$ | 2.00 |
| $f_2(x)$ | 2.00 |
| $f_3(x)$ | 2.00 |
| $f_4(x)$ | 2.00 |
| $f_5(x)$ | 1.37 |
| $f_6(x)$ | 1.50 |

Table 4

**Remarks**

- All integrals except those of functions $f_5$ and $f_6$ to the exact value at second order with respect to the subinterval length. Indeed, the integrand functions are all smooth in the integration interval. This assures by expanding with Taylor that the truncation error is quadratic in $h$ as $h$ goes to zero.

- $f_5$ is not differentiable in 0, hence the linear interpolation fails at producing a "good" approximation of the function in $x = 0$. We can try to estimate the truncation error analitically. We will consider only the subinterval $[0, \epsilon]$, since in all others the function is smooth and will converge

quadratically. We have:

$$
\begin{aligned}
R &= \int_0^\epsilon \sqrt{x}\log(x)\mathrm{d}x - \frac{\epsilon^{\frac{3}{2}}}{2}\log(\epsilon) = \\
&= \left[\frac{2}{3}\epsilon^{\frac{3}{2}}\log(x)\right]_0^\epsilon - \frac{2}{3}\int_0^\epsilon \sqrt{x}\mathrm{d}x - \frac{\epsilon^{\frac{3}{2}}}{2}\log(\epsilon) = \\
&= \epsilon^{\frac{3}{2}}\log(\epsilon)\left(\frac{1}{6}\log(\epsilon) - \frac{4}{9}\right)
\end{aligned}
$$

Hence the truncation error is $O(h^{\frac{3}{2}}\log(h))$. Note that as h goes to zero, $\log(|R|) \sim m\log(\epsilon)$, hence the estimated $m$ value should be asymptotically 1.5.

- $f_5$ is not differentiable in 1. As in previous point we can try to estimate the truncation error analitically:

$$
\begin{aligned}
R &= \int_{1-\epsilon}^1 \sqrt{1-x^2}\mathrm{d}x - \frac{\epsilon}{2}\sqrt{1-(1-\epsilon)^2} = \\
&= \int_0^\epsilon \sqrt{2y-y^2}\mathrm{d}y - \frac{\epsilon}{2}\sqrt{2\epsilon-\epsilon^2} = \\
&= \int_0^\epsilon \sqrt{2y}(1+O(y))\mathrm{d}y - \frac{\epsilon}{2}\sqrt{2\epsilon}(1+O(y)) = \\
&= \frac{2\sqrt{2}}{3}\epsilon^{\frac{3}{2}} - \frac{\sqrt{2}\epsilon^{\frac{3}{2}}}{2} + O(y^{\frac{5}{2}}) = \\
&= \frac{\sqrt{2}}{6}\epsilon^{\frac{3}{2}} + O(\epsilon^{\frac{5}{2}})
\end{aligned}
$$

Hence the truncation error is $O(h^{\frac{3}{2}})$, as hinted in the Results paragraph.

- Since the trapezoidal rule approximates the integrand function with linear interpolations, intervals in which the function is convex produce underestimates, while intervals where the function is concave produce overstimated integrals. In fact, $f_1$, $f_2$ and $f_5$ are convex within the bounds of integration: the error is negative. On the other hand, $f_3$ and $f_6$ are concave: the error is positive.

### 5.1.2 Simpson's rule

Exactly like in previous section, we test our implementation of Simpson's rule on the functions in 5.1.

In ADD FIGURE AND REFERENCES we show again the truncation error as a function of the number of subintervals used.

In Table 5 we report the estimated values for the orders of $m$.

The first four functions are smooth in the integration interval, hence for Simpson's rule fourth-order convergence is guaranteed.

Calculations analogous to those in previous section would show that the truncation errors of $f_5(x)$ and $f_6(x)$ are, respectively, $O(h^{\frac{3}{2}\log(h)}$ and $O(h^{\frac{3}{2}}$.

One might point out that the estimated value of $m$ for function $f_5(x)$ is very close to $\frac{7}{2}$. It is a coincidence. If we changed the program `simpson_rule.c` in order to remove some initial points from the fit, we would see that the value of $m$ would get closer and closer to 1.5.

| $f$ | m |
|---|---|
| $f_1(x)$ | 4.00 |
| $f_2(x)$ | 4.00 |
| $f_3(x)$ | 4.00 |
| $f_4(x)$ | 4.01 |
| $f_5(x)$ | 1.40 |
| $f_6(x)$ | 1.50 |

Table 5

## 5.2 Gauss-Legendre rule

### 5.2.1 Nodes and weights of Gauss-Legendre rule

In this section we designed and tested some routines for evaluating Legendre polynomials, their roots and the weights in Gauss-Legendre integration.

**Results** In Table 6 we report the values of roots and weights Legendre polynomials with degree 2–4 that we found with our routines. In the program `gauss_legendre_weights.c` we also compare them to the exact values, to which they agree with an error lower than the tolerance requested in the rootfinding routine.

In ADD FIGURE AND PLOT we also show the plot of the first 6 Legendre polynomials calculated with our routines.

| Degree | Root index | Root value | Weight |
|---|---|---|---|
| 2 | 1 | 0.577 | 1.000 |
| | 2 | -0.577 | 1.000 |
| 3 | 1 | 0.775 | 0.556 |
| | 2 | 0.000 | 0.889 |
| | 3 | -0.775 | 0.556 |
| 4 | 1 | 0.861 | 0.348 |
| | 2 | 0.340 | 0.652 |
| | 3 | -0.340 | 0.652 |
| | 4 | -0.861 | 0.348 |

Table 6

**Remarks**

- For evaluating the $n$th Legendre polynomial for $L_n(x)$, the values for all lower degrees must be computed recursively for the same $x$. As a micro-optimization, these are stored inside an `static` array. Since `static` arrays in C are conserved between function calls, this saves time if $L_m(x)$ for the same $x$ is requested in the following function call. Our routine will compute values of $L_n(x)$ only if they are not already stored, otherwise it will use the values already present to evaluate a higher degree polynomial. The array size count resets only when the caller asks for a different value of $x$.

- For this reason, evaluating derivatives is extremely cheap and justifies the use of Newton method for finding the roots. The algorithm has complexity $O(nk)$, where $n$ is the degree of the polynomial and $k$ is the number of steps in Newton method. It should be a low number, reasonably independent from $n$, and most likely less than 10.

### 5.2.2 Integrals with Gauss-Legendre rule

We now test our routine for calculating integrals with Gauss-Legendre rule and we study the convergence rate. We compute the integrals of the functions:

1) $\int_{-1}^{1} e^{-4x}\mathrm{d}x$

2) $\int_{-1}^{1} e^{-9x^2}\mathrm{d}x$

3) $\int_{-1}^{1} \operatorname{sech}(x)\mathrm{d}x$

4) $\int_{-1}^{1} \frac{1}{1+9x^2}\mathrm{d}x$

5) $\int_{\pi/2}^{\pi} x^2 \sin(8x)\mathrm{d}x$

In ADD FIGURE AND REFERENCE we plot the error in integral estimation as a function of the degree of the interpolant polynomial.

The convergence is very fast and is exponential or superexponential. But why do some functions appear to converge much faster than others?

The remainder term in Gauss-Legendre rule is guaranteed to be:

$$R_P[f] = \frac{2^{2n+1}(n!)^4}{(2n+1)[(2n)!]^3} f^{(2n)}(\xi) \qquad \xi \in (-1,1) \tag{3}$$

where $f$ is the integrand function after we have changed variables in the integration interval.

The function $f_4 = \frac{1}{1+9x^2}$ converges as the slowest. In fact:

$$\begin{aligned}
\frac{\mathrm{d}^n}{\mathrm{d}x^n}\frac{1}{1+9x^2} &= 3^n \frac{\mathrm{d}^n}{\mathrm{d}y^n}\frac{1}{1+y^2} = \\
&= \frac{(-1)^n 3^n n!}{2i}\left(\frac{1}{(y-i)^{n+1}} - \frac{1}{(y+i)^{n+1}}\right) \\
&= \frac{(-1)^n 3^n n!}{2i(y^2+1)^{\frac{n+1}{2}}}\left(e^{i(n+1)\theta} - e^{-i(n+1)\theta}\right) \\
&= \frac{(-1)^n 3^n n!}{(y^2+1)^{\frac{n+1}{2}}}\sin\left((n+1)\arctan\left(\frac{1}{y}\right)\right) \\
&= \frac{(-1)^n 3^n n!}{(9x^2+1)^{\frac{n+1}{2}}}\cos\left((n+1)\arctan 3x - \frac{n\pi}{2}\right)
\end{aligned}$$

Since we took $\theta = \arctan(\frac{1}{y})$, this technically would be valid only if $\theta \in [-\frac{\pi}{2}, \frac{\pi}{2}]$, i.e. if $y = 3x > 0$, but this will not matter since all derivatives are either even or odd, and we are interested only in the absolute value of them. Plugging this into Equation 3 we find, after some simplifications:

$$R_P[f] \approx k \left(\frac{3}{2}\right)^{2n} (-1)^n \frac{\cos\left((2n+1)\arctan(3\xi)\right)}{(9\xi^2+1)^{n+\frac{1}{2}}} \qquad \xi \in (-1,1)$$

where $k$ is a constant factor. This... is not great help, since, depending on the value that $\xi$ takes, the above expression can be, in absolute value, anything between 0 and $\left(\frac{3}{2}\right)^{2n}$. The point is that in this case the remainder theorem does not guarantee a superexponential convergence as we will see in other cases, and it has to do with the the magnitude of the derivatives of $\frac{1}{1+3x^2}$.

Nonetheless, $f_4$ is analytic and bounded in a Bernstein ellipse $E_\rho$ with $\rho < 3$. Therefore convergence is guaranteed to be exponential, but it is not faster than that, as we can see from our plot.

With $f(x) = e^{-4x}$ the calculations are easier and convergence is much faster. We have $f^{(n)}(x) = (-4)^n e^{-4x}$. Plugging into equation 3 we find the remainder:

$$\begin{aligned} R_P[f] =& \approx \frac{\sqrt{n}e^{-4\xi}}{(n!)^2} \qquad \xi \in (-1,1) \\ &\leq k \frac{\sqrt{n}e^4}{[(n)!]^2} \end{aligned}$$

With $k$ constant. That is, we have squared-factorial convergence guaranteed!

## 5.3 Double exponential rule

### 5.3.1 Comparison of double exponential and Gauss-Legendre rules

We now test our routine for the double-exponential rule on the integrals:

1) $\int_0^1 \sqrt{x}\log(x)\mathrm{d}x = -\frac{4}{9}$

2) $\int_0^1 \sqrt{1-x^2}\mathrm{d}x = \frac{\pi}{4}$

3) $\int_1^0 (\log x)^2 \mathrm{d}x = 2$

4) $\int_{\pi/2}^{\pi} \log(\cos x)\mathrm{d}x = -\frac{\pi}{2}\log 2$

5) $\int_{\pi/2}^{\pi} \sqrt{\tan x}\mathrm{d}x = \frac{\pi}{\sqrt{2}}$

and we study the the error as a function of $n$, comparing it with that of the Gauss-Legendre rule. Here $n$ is the number of nodes in the Gauss-Legendre rule and the number of subintervals in the double-exponential rule.

In ADD FIGURE AND REFERENCES we plot the error as a function of $n$.

**Remarks**

- Our routine is highly modular in the sense that it is possible to inject into `double_exp_base` the double exponential diffeomorphism suited for the needs of the caller.

In `intg_double_exp` it is possible to choose between four different diffeomorphisms that we implemented, one for bounded intervals, the other two for the whole real line and half line.

- The double-exponetial rule maps the integration interval $I$ into the real line, and calculates the resulting integral with the trapezoidal rule. Determing where to truncate the latter integral requires some care.

  Let $t_M > 0$ so that the transformed integral is truncated to the interval $[-t_M, t_M]$. In order to determine $t_M$, our routine sets initially its value to 0.1, and then adds 0.1 until both $|g(t_M)|$ and $|g(-t_M)|$ are lower than a certain `tol`, where $g$ is the integrand function of the transformed integral. We also check against `NaN` that could be returned by some functions on singular points.

  Since we are dealing with double exponential functions, which grow extremely fast, it is necessary to safeguard for overflow and underflow cases. Even for just $t_M = 6.6$, $e^{e^{t_M}} \approx 10^{319}$, which overflows double precision floating representation. The sinh-tanh quadrature rule is even more delicate. As t becomes "big" (when dealing with double exponentials "big" numbers are reached very fast...) we have:

  $$\phi(t) = \tanh\left(\frac{\pi}{2}\sinh t\right) \approx 1 - 2e^{-\frac{\pi}{2}e^t}$$

  when $t_M = 3.2$, $2e^{-\frac{\pi}{2}e^{t_M}} \approx 3.7 \times 10^{-17} < \epsilon_{\text{mach}}$, therefore we have an underflow and $\psi(t_M) = 1$. A proper and implementation should take into account the possibilities of underflow and overflow for every diffeomorphism used. However, due to the overall design and time constraints, we only check the overflow for integrals over real line, and we leave to the caller the responsibility to guard against overflows in the tanh-sinh case, by adjusting the `tol` required and therefore allowing the routine to settle on a lower value of $t_M$.

- All the integrand functions of the integrals considered are either infinite or non differentiable at the bounds of integration. Unsurprisingly, convergence is slow with the Gauss-Legendre rule. In fact, all functions have unbounded derivatives, and functions 3-5 are not even bounded themselves. Although the integrals appear to converge nonetheless, it is yet very slow and likely algebraic.

  On the contrary, the approximations with double-exponential quadrature rule converge exponentially with $n$. For this rule, fast convergence is guaranteed even if the integrand is not analytic at the limits of integration, as it is the case for the integrand functions above.

- The only exceptions appear to be $f_3$ and $f_5$, which, after a phase of exponential convergence, the errors appear to stop shrinking and start growing slowly at $10^{-8}$ and $10^{-13}$. The problem here is not the theoretical convergence of double-exponential rule, but the truncation error due to considering the transformed integral only in $[-t_M, t_M]$. In these two cases, indeed, we had to lower manually the tolerance `tol` in the routine for determining $t_M$, as otherwise it would push for a value of $t_M$ too high, resulting in an underflow. Setting a high tolerance in truncation determination has of

naturally the effect of adding a constant error which cannot be lowered by adding more points.

A smart way to achieve more accuracy for the integral of function $f_5$ is to use a change of variables:

$$\int_0^{\frac{\pi}{2}} \sqrt{\tan(x)}\mathrm{d}x = \int_0^{\infty} \frac{\sqrt{y}}{1+y^2}\mathrm{d}y$$

As observed above, the double-exponential rule for infinite intervals is more safeguarded from the risk of overflow. A quick function call in the program shows indeed that the estimate reaches $\epsilon_{\text{mach}}$ levels of precision by using just $n = 36$ nodes.

### 5.3.2 Improper integrals with double exponential rule

We now test our implementation of the double-exponential rule on integrals over infinite intervals, and study the convergence:

1) $\int_{-\infty}^{\infty} \frac{1}{1+x^2+x^4}\,\mathrm{d}x = \frac{\pi}{\sqrt{3}}$

2) $\int_{-\infty}^{\infty} e^{-x^2}\cos(x)\,\mathrm{d}x = e^{-1/4}\sqrt{\pi}$

3) $\int_{-\infty}^{\infty} (1+x^2)^{-2/3}\,\mathrm{d}x = \frac{\sqrt{\pi}\,\Gamma(1/6)}{\Gamma(2/3)}$

4) $\int_{0}^{\infty} \frac{1}{1+x^2}\,\mathrm{d}x = \frac{\pi}{2}$

5) $\int_{0}^{\infty} \frac{e^{-x}}{\sqrt{x}}\,\mathrm{d}x = \sqrt{\pi}$

For integrals 1-3 we use as change of variables:

$$I = \int_{-\infty}^{\infty} f(x)\,\mathrm{d}x \qquad\qquad x = \sinh\left(\frac{\pi}{2}\sinh(t)\right)$$

For integral 4 we use:

$$I = \int_{0}^{\infty} f(x)\,\mathrm{d}x \qquad\qquad x = \exp\left(\frac{\pi}{2}\sinh(t)\right)$$

For integral 5 we use:

$$I = \int_{0}^{\infty} f(x)e^{-x}\,\mathrm{d}x \qquad x = \exp\left(t - \exp(-t)\right)$$

after putting $f(x) = \sqrt{x}$.

In ADD FIGURE AND REFERENCES we show the plot of the absolute value of the error as a function of the number of function evaluations $n$. Despite oscillations, the errors appears to decay linearly with $n$ in the semi-log scale, for all the above integrals. We then conclude that it shrinks exponentially. The oscillations are due to the fact that the transformed integrand functions have varying concavity, producing therefore both positive and negative errors that can "accidentally" cancel out, resulting in a numerical estimate more precise than the subsequent estimate with more nodes.

# 6 Ordinary differential equations

## 6.1 Euler's method

In the following paragraphs we study the Euler's method for solving Initial Value Problems (IVP) of ODEs. Given a diffential equation in the form $\mathbf{u}' = \mathbf{f}(\mathbf{u}, t)$, forward Euler's method approximates the solution at next timestep as:

$$\mathbf{u}(t_{i+1}) = \mathbf{u}(t_i) + \mathbf{f}(\mathbf{u}(t_i), t_i)$$

### 6.1.1 First order application

We first test our routine for Euler's method on the following first order scalar IVPs:

1. $u' = -2tu, \;\; 0 \leq t \leq 2, \;\; u(0) = 2;$
2. $u' = u + t, \;\; 0 \leq t \leq 1, \;\; u(0) = 2;$
3. $(1 + t^3)uu' = t^2, \;\; 0 \leq t \leq 3, \;\; u(0) = 1;$

In ADD FIGURE AND REFERENCE we show the solutions calculated with $n = 320$ timesteps, after normalizing their domains to [0,1] in order to fit them all in one plot.

**Study of convergence** In ADD FIGURE AND REFERENCES we show the error in our numerical estimates as a function of the number of timesteps utilised. On the y axis we have drawn the error calculated in sup-norm as well as the error at the last time step. Since the the number of timesteps is inversely proportional to size of the timesteps, in Graph ADD REFERENCE we can clearly see first order convergence of the error $O(h)$, where $h$ is the length of the timestep, by noting that the error lines are parallel to a first order convergence reference line.

### 6.1.2 Second order application

We first test our routine for Euler's method on the following second order scalar IVPs:

1. $y'' + 9y = \sin(2t), \;\; 0 < t < 2\pi, \;\; y(0) = 2, \;\; y'(0) = 1$
2. $y'' - 4y = 4t, \;\; 0 < t < 1.5 \;\; y(0) = 2 \;\; y'(0) = -1$
3. $y'' + 4y' + 4y = t, \;\; 0 < t < 4 \;\; y(0) = 1 \;\; y'(0) = 3/4$

Each can be reduced to a first order IVP by introducing the variables $u_0 = y$, $u_1 = y'$ and the equation $u_0' = u_1$.

In Graphs ADD FIGURES AND REFERENCES we show the functions thus calculated, their derivatives and the deviations with the exact solutions.

## 6.2 Runge-Kutta methods

### 6.2.1 Improved Euler's and Runge-Kutta fourth order methods

We now test our routine for Improved Euler (IE2) and Runge-Kutta 4 (RK4) on the IVP:

$$u' = -2tu,\ u(0) = 2,\ t \in [0, 2]$$

and study the convergence.

The plot of the solution can be found in ADD REFERENCE, since the same problem was integrated in 6.1.1. In ADD FIGURE AND REFERENCE we plot the maximum error between the numerical solution and the exact solution as a function of the number of function evaluations. Even though RK4 reuquires twice as many function evaluations as IE2, RK4 is way more accurate even when taking this into account, thanks to fourth order convergence $O(h^4)$ of RK4's error as stepsize $h$ becomes small.

### 6.2.2 Non-linear equation application of Runge-Kutta

We now use RK4 to study the following two nonlinear systems of two equations of first order:

$$\textbf{(a)} \qquad \begin{aligned} x'(t) &= -4y + x(1 - x^2 - y^2)\,, \\ y'(t) &= 4x + y(1 - x^2 - y^2)\,, \end{aligned} \tag{4}$$

we test initial conditions $[x(0), y(0)] = [0.1, 0]$ and $[x(0), y(0)] = [0, 1.9]$;

$$\textbf{(b)} \qquad \begin{aligned} x'(t) &= -4y - \tfrac{1}{4}x(1 - x^2 - y^2)(4 - x^2 - y^2)\,, \\ y'(t) &= 4x - \tfrac{1}{4}y(1 - x^2 - y^2)(4 - x^2 - y^2)\,, \end{aligned}$$

in this system we test initial conditions $[x(0), y(0)] = [0.95, 0]$, $[0, 1.05]$, and $[-2.5, 0]$.

In ADD FIGURES AND REFERENCES and ADD FIGURE AND REFERENCES we plot the numerical solutions for time $t \in [0, 10]$ and $n = 1000$ timesteps.

**Validity of numerical solutions** Since we have no analytical solution to compare our result with, we have to estimate the error in some other way. The global error of our numerical solution should be $O(h^4) = O(n^{-4})$, where $h$ is the stepsize and $n$ is the number of points used. If we rerun our resolver with $2n = 2000$ points, the error should be around 1/16 times smaller, hence negligible for rough calculations. Therefore we can use as a measure of the order of magnitude of the error the difference between the numerical solutions with 1000 and 2000 point in the times in which they overlap (i.e. every value of the $n = 1000$ and every two values of the $n = 2000$ solution). In Table 7 we report for each curve considered the errors thus calculated. To calculate the error between two points $(f_x(t_i), f_y(t_i))$ and $(f'_x(t_i), f'_y(t_i))$ we just consider $\epsilon = \max(|f_x(t_i) - f'_x(t_i)|, |f_y(t_i) - f'_y(t_i)|)$

This suggests that the errors are controlled in the range $10^{-10} - -10^{-6}$, which more than enough precision for our limited study.

| System | $x_0$,$y_0$ | $\epsilon_f$ | $\epsilon_{\max}$ |
|---|---|---|---|
| a | 0.1,0 | $5.12 \times 10^{-7}$ | $6.89 \times 10^{-7}$ |
| | 0,1.9 | $5.62 \times 10^{-7}$ | $7.56 \times 10^{-7}$ |
| b | 0.95,0 | $1.26 \times 10^{-10}$ | $1.13 \times 10^{-7}$ |
| | 0,1.05 | $1.30 \times 10^{-6}$ | $1.56 \times 10^{-6}$ |
| | -2.5,0 | $1.29 \times 10^{-6}$ | $1.54 \times 10^{-6}$ |

Table 7: $\epsilon_f$ is the error at last timestep, while $\epsilon_{\max}$ is the maximum of the errors.

**Analysis of the dynamics of the systems** The two systems present similar dynamics, in the sense that in both we can see that the trajectories of motion are spyrals centered in the origin.

For system a), the spyrals seem to converge asymptotically to a closed circular orbit. This motivates the following simple calculation using the equations of motion 4:

$$\begin{aligned}\frac{\mathrm{d}x^2+y^2}{\mathrm{d}t} &= 2xx' + 2yy' = \\ &2x(-4y + x(1-x^2-y^2)) + 2y(4x + y(1-x^2-y^2)) = \\ &(x^2+y^2)(1-x^2-y^2) = r^2(1-r^2)\end{aligned}$$

which proves that the solutions of motion move away from the origin when they lie inside the unit circle, and towards it when they are outside. If a point of motion lies on the unit cirlce, then the equation of motion reduce to:

$$\begin{aligned}x &= -4y \\ y &= 4x\end{aligned}$$

which are the equation of motion of a particle moving on a circle in uniform circular motion. Since two solutions cannot intersect in the phase-space, all solutions not lying on the unit circle are bound to spyral indefinitely towards it, without ever touching it.

Equation b) behaves similarly, except that there are two circle on which the motion is uniformly circular, $r = 1$ and $r = 2$. In fact, we have here:

$$\frac{\mathrm{d}r^2}{\mathrm{d}t} = -\frac{1}{2}r^2(1-r^2)(4-r^2)$$

Which is negative when $r < 1$ or $r > 2$ and is positive when $1 < r < 2$. Hence solutions inside the unit circle converge asymptotically to the center, while solutions outside convergence to the circle of radius 2.

### 6.2.3 Predator-prey model

We now apply our RK4 routine to solve find a solution of the predator–prey model:

$$\begin{aligned}\frac{\mathrm{d}y}{\mathrm{d}t} &= y(1-\alpha y) - \frac{yz}{1+\beta y}, \\ \frac{\mathrm{d}z}{\mathrm{d}t} &= -z + \frac{yz}{1+\beta y},\end{aligned}$$

where we used as parameters $\alpha = 0.1$ and $\beta = 0.25$. In ADD FIGURE AND REFERENCE we show the resulting curves of motion drawn in phase-space for a few starting values in the time frame $t \in [0, 60]$.

It is easy to see that a nontrivial equilibrium point is:

$$(x*, g*) = \left(\frac{1}{1-\beta}, \frac{1}{(1-\beta)^2}(1-\beta-\alpha)\right) = \left(\frac{4}{3}, \frac{52}{45}\right)$$

which appears to be unstable, since a solution starting from a point near it spyrals outwards away from it.

After long times the curves seem to converge along a closed curve. We have tried to draw it by computing points up to time $t = 660$.

Additionally, in Graph ADD FIGURE AND REFERENCE we show the evolution of prey and predators over time. As we expect from prey-predators model, the two variables oscillate almost periodically.

# 7 Schrödinger equation

In the following paragraphs we solve the one-dimensional Time-Dependant Schrödinger Equation (TDSE) and Time-Independant Schrödinger Equation (TISE).

To discretize the Hamiltonian, we first discretize space in equally spaced nodes and use the following finite difference approximation for the second derivative:

$$\left.\frac{d^2\psi}{dx^2}\right|_{x_j} \approx \frac{-\psi_{j+2} + 16\psi_{j+1} - 30\psi_j + 16\psi_{j-1} - \psi_{j-2}}{12(\Delta x)^2} + \mathcal{O}(\Delta x^4)\,. \tag{5}$$

We use Dirichlet boundary condition, that is, we consider fixed at 0 the wavefunction outside the simulation interval. This is equivalent to putting the system inside an infinite potential well.

## 7.1 Time-dependant Schrödinger equation

For solving TDSE, we use a fourth-order Runge-Kutta algorithm specialized for time-independant potentials. This time discretization is equivalent to a fourth order expansion of time evolution operator. In natural units:

$$\begin{aligned}
\langle\psi(t+\mathrm{d}t)| &= U(\mathrm{d}t)\,\langle\psi(t)| \\
&= e^{-iH\mathrm{d}t}\,\langle\psi(t)| \\
&= \left(\mathrm{I} + i\mathrm{H}\mathrm{d}t - \frac{1}{2}\mathrm{H}^2\mathrm{d}t^2 - i\frac{1}{6}\mathrm{H}^3\mathrm{d}t^3 + \frac{1}{24}\mathrm{H}^4\mathrm{d}t^4\right)\langle\psi(t)|
\end{aligned}$$

In all simulations, we use a Gaussian wave packet:

$$\psi(x,0) = \frac{1}{(2\pi\sigma^2)^{1/4}} e^{-(x-x_0)^2/4\sigma^2} e^{ik_0 x}$$

with Fourier transform:

$$\psi(k,0) = \left(\frac{2\sigma^2}{\pi}\right)^{1/4} e^{-(k-k_0)^2\sigma^2} e^{-ix_0(k-k_0)}$$

### 7.1.1 Gaussian packet in an infinite well

We first test our algorithm by studying time evolution of a free Gaussian wave packet with average momentum zero. The result can be visualised in the `gifs` folder inside the `report` directory under the name `schrodinger_gaussian.gif`. Considering only the square modulus of the wave function, as expected the packet first spreads out, its peak falling, while its tails expanding. Then it gets reflected back when it hits the walls of the well, giving rise to standing waves.

Note that in our test $\Delta x = 0.1$ and $\Delta t = 0.01$ satisfies the stability condition for the free case

$$\Delta t \lesssim= \frac{m(\Delta x)^2}{\hbar}$$

Raising $\Delta t$ even so slightly as to 0.02 causes the solution to explode.

**Norm conservation** Schrödinger equation preserves probabilities over time, and we want to check how good our numerical solver reproduces this property. To evaluate the square norm we just integrate the square modulus of the wave function with a midpoint rule, considering even the values at the endpoints since the function should ideally continue beyond them if we were not to truncate space.

First we evaluate the square norm of the packet at time $t = 0$. We find 0.995. This is sensibly different than the expected value 1. Is it because the quadrature rule that we used was not accurate enough with these many grid points or we cut off the tails of the gaussian? Definitely the latter, as repeating the calculation with a grid 100 times larger yields a square norm value of 0.99995.

Using $|\langle\psi|\,|^2 = 0.995$ as norm reference value, we can compare at each time the difference between the square norm at that time and the reference value. At time $t = 20$ in time evolution the difference is maximum at 0.006, i.e. 0.6%. Our numerical solver then preserves quite well probabilities for the parameters and time scales provided.

### 7.1.2 Moving gaussian packet

We test again our routine on a gaussian wavepacket, this time with nonzero starting average momentum $k_0$. The result can be again visualised in the `gifs` folder inside the `report` directory under the name `schrodinger_gaussian2.gif`. The animation shows clearly two concurrent phenomena: first, the packet as a whole is moving towards the right. In fact, its group velocity is different than zero, since we prepared nonzero average momentum. Second, the packet appears to flatten and spread like in the zero momentum case.

As in the previous example, the norm is roughly conserved with a loss of probability at last time of $\approx 0.5\%$.

### 7.1.3 Finite barrier scattering

OPTIONAL: ADD THE STUDY OF R AND T AS FUNCTIONS OF $V_0$ AND $A$

We now seek to study scattering from a finite square barrier:

$$V(x) = \begin{cases} V_0 & -a \leq x \leq a \\ 0 & \text{elsewhere} \end{cases}$$

with $V_0 > 0$ and $a > 0$.

For $V_0 = 2$ and $a = 1$ the result can be visualised in the `gifs` folder inside the `report` directory under the name `finite_barrier_gaussian.gif`.

In the animation we see that the wave packet seems to divide into two separate chunks: one proceeding to the right, having passed through the barrier; the other being reflected in the opposite side. With initial conditions requested in [Dal25], we find the reflection and transmission coefficients:

$$R = 0.690 \quad T = 0.303$$

Summed up, these yield $R + T = 0.99344 \approx 1$, which differs from the norm calculated at time 0 inside the simulation interval by 0.0061.

**Study of the behavious of $R$ and $T$** The dependance of $R$ and $T$ on the parameters $V_0$ and $a$ is not simple. For an eigenfunction of definite energy $E < V_0$, the reflection and transmission coefficients are [CDL20, Complement $\mathrm{N_{III}}$]:

$$T = \frac{1}{\cosh^2(2a\sqrt{2(V_0 - E)}) + \left(\frac{k^2 - k_0^2}{2kk_0}\sinh(2a\sqrt{2(V_0 - E)})\right)^2}$$

$$R = \frac{\left(\frac{k_0^2}{4k\rho}\sinh(2a\sqrt{2(V_0 - E)})\right)^2}{\cosh^2(2a\sqrt{2(V_0 - E)}) + \left(\frac{k^2 - k_0^2}{2kk_0}\sinh(2a\sqrt{2(V_0 - E)})\right)^2}$$

### 7.1.4 Pöschl-Teller potential

We now apply our numerical solver to study the scattering from the Pöschl-Teller potential:

$$V(x) = \frac{V_0}{\cosh^2\left(\frac{x}{\omega}\right)}$$

We tested it with different combinations of the starting parameters. The results can be visualised in the `gifs` folder inside the `report` directory under the name `poschl_teller_gaussian_X.gif`, with X being the codename given to that simulation. In Table 10 we include some of the starting parameters and the values of the reflection and transmission coefficients $R$ and $T$.

| Name | $V_0$ | $\omega$ | $R$ | $T$ |
|---|---|---|---|---|
| Positive potential | 2 | 1 | 0.428 | 0.565 |
| Negative potential | -2 | 0.5 | 0.147 | 0.835 |
| Special | -12 | 0.5 | 0.000 | 0.982 |

Table 10

A positive potential seems to favour reflection, while a negative potential transmission. This should be no surprise, since a positive potential acts more like a barrier, while a negative one like a well.

The Pöschl-Teller potential is special because it can be solved exactly. Remarkably, when the condition $V_0 = -l(l+1)/2\omega^2$ is satisfied, scattering through the potential produces total transmission of the incident wave []. The simulation above marked as "special" shows this precise behaviour.

## 7.2 Time-independant Schrödinger equation

For TISE, we solve a discretized version of the eigenvalue problem:

$$\mathrm{H}\,|\psi\rangle = E\,|\psi\rangle$$

As an eigenvalue solver, we chose to implement a version of QR with Wilkinshon shifts described in Trefethen and Bau [TB97, Lecture 29. QR Algotithm with shifts] specialized for pentadiagonal matrices, as the one produced by the discretization in Equation 5.

We outline here the procedure:

1. With Householder reflectors we reduce H in Hessenberg ( tridiagonal) form [TB97, Lecture 26. Reduction to Hessenberg or Tridiagonal Form]. This step requires applying repeatedly Householder reflectors,represented by orthogonal matrices $Q_i$, to H. This is equivalent to applying a matrix $\mathbf{Q}^\mathsf{T}$ to H. This produces the decomposition $\mathrm{H} = Q_0\mathrm{H}_0$, where $\mathrm{H}_0$ is tridiagonal and $Q_0$ is orthogonal. We can reconstruct $Q_0$ by storing Householder reflectors.

2. We calculate Wilkinon shift $\mu$, then QR decompose $QR = \mathrm{H}_0 - \mu\mathrm{I}$ using a routine specialized for tridiagonal matrices inspired by Householder reflectors. $Q$ is accumulated on $Q_0$ inside the routine in order to avoid a costly matrix multiplication between $Q$ and $Q_0$

3. A new matrix $\mathrm{H}_1 = RQ + \mu\mathrm{I}$ is assembled.

4. Deflation is applied. In our routine we just check if the entries adjacent to the bottom right entry of $\mathrm{H}_1$ are 0 within a small tolerance. If so, they are set to 0 and the matrix is considered to be one dimension lower in future iterations.

5. Steps 2–4 are repeated until the matrix is fully diagonal.

If $n_i$ is the dimension of the square tridiagonal matrix $H_i$ at iteration $i$ (after considering deflation), then step 2 can be performed in $O(n_i^2)$. This cost could be further improved to $O(n_i)$ by fusing Step 2 and 3 and avoiding to calculate explicitly the full matrix $Q$. Step 3 can be performed just in $O(n_i)$, thanks to the fact that $R$ is tridiagonal.

The whole algorithm converges in around $O(n_0)$ iterations, where $n_0$ is the dimension of $H_0$, but this could be much improved with a more sofisticated deflation algorithm.

Right now the bottleneck of our implementation is actually Step 1, since we have not written a routine specialized for reducing pentadiagonal matrices into tridiagonal matrices, due to time constraints. As it stands, it is $O(n_0^3)$, as it is the whole algorithm overall.

**Results** By using 200 points and a lattice spacing of 0.05, we find as energies:

| Exact $E$ | Numerical $E$ | Difference |
|---|---|---|
| -9.18 | -9.21 | $3.3 \times 10^{-2}$ |
| -6.78 | -6.91 | 0.13 |
| -3.05 | -3.30 | 0.25 |

Table 10

With errors of the order of 10% the values are not very accurate. By using 1000 points and a lattice spacing of 0.01, we can do much better:

| Exact $E$ | Numerical $E$ | Difference |
|---|---|---|
| -9.18 | -9.19 | $6.7 \times 10^{-3}$ |
| -6.78 | -6.80 | $2.5 \times 10^{-2}$ |
| -3.05 | -3.10 | $4.9 \times 10^{-2}$ |

Table 10

Now the error is around, at most, 1%. Note, however, that with this many points the algorithm is 125 slower, because of $O(n^3)$ scaling. A $O(n^2)$ algorithm, entirely feasible as hinted above, would improve dramatically the number of points realistically usable.

In ADD FIGURE AND REFERENCES we show the plot of the bound states thus found.

# Bibliography


[] *Pöschl–Teller potential*. Wikipedia. URL: https://en.wikipedia.org/wiki/P%C3%B6schl%E2%80%93Teller_potential (visited on 11/10/2026).

[CDL20] Claude Cohen-Tannoudji, Bernard Diu, and Franck Laloë. *Quantum Mechanics*. 2nd ed. Vol. 1. Wiley-VCH, 2020. ISBN: 9783527345533.

[Dal25] Mattia Dalla Brida. "Lecture notes in Computational Physics". 2025.

[TB97] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997. ISBN: 0898713617.